

CSCE 411: Design and Analysis of Algorithms

Lecture 1: Intro, Asymptotic Runtimes, Divide and Conquer

Date: August 25, 2026

Nate Veldt, updated by Samson Zhou

Course Logistics

- Read section 2.3, chapter 4, and chapter 30 for first week of classes.
- Read (or skim) chapters 1-3 to ensure familiarity with prerequisites
- HW 1, intro video, and syllabus quiz are due Fri, Sep 4.

1 Computational Problems and Algorithms

Definition 1. A _____ is a general task defined by a specific type of _____ and an explanation for a desired _____

A specific case of the problem is called an _____

Example 1. Sorting

Input: A sequence of n numbers: $a_1, a_2, \dots, a_n$

Output: A permutation σ of the input sequence so that

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}$$

An instance of this problem is the sequence

Example 2. Min element

Input: An array of n numbers: $[a_1, a_2, \dots, a_n]$

Output: The smallest element in the array and its index.

Definition 2. An **algorithm** is a computational procedure that

- _____
- _____

An **algorithm** is said to be **correct** if it _____.

2 Asymptotic Runtime Analysis (Chapter 3)

2.1 Rules for runtime analysis

- n denotes the size of the input
- Each basic operation takes constant time
- We focus on the _____ runtime
- We only care about the _____ of the runtime

2.2 An initial example

Question 1. Given an $n \times n$ matrix A , what is the runtime of summing the upper triangular portion using the following algorithm? (same answers).

```
sum = 0
for i = 1 to n do
    for j = i to n do
        sum = sum + aij
    end for
end for
Return sum
```

- A** $O(1)$
- B** $O(n)$
- C** $O(n^2)$
- D** It depends
- E** Other
- F** Don't know, I need a reminder for how this works.

2.3 Formal Definitions

Let n be input size, and let f and g be functions over $\mathbb{N}$.

Definition 3. Big O notation.

A function $g(n) = O(f(n))$ (we say, “ g is big-O of $f(n)$ ”) means:

Definition 4. Big Ω notation.

$g(n) = \Omega(f(n))$ means:

Definition 5. Θ notation.

$g(n) = \Theta(f(n))$ means:

3 How to Present an Algorithm

Presenting and analyzing can be broken up into four steps.

1. **Explain:** the approach in basic English
2. **Pseudocode:** for formally presenting the algorithmic steps
3. Prove: the **correctness**
4. Analyze: the **runtime complexity**

As a rule it's a good idea to go through all steps when presenting an algorithm. Sometimes we will focus more on just a subset of these (e.g., you may be asked to prove a runtime complexity of an algorithm on a homework but not a correctness proof).

We will go through all four steps when we present the *merge sort* algorithm.

4 The Divide and Conquer Paradigm

The divide and conquer paradigm has three components:

- **Divide:**
- **Conquer:**
- **Combine:**

Example: Mergesort (Textbook, Chapter 2.3, 4) Given n numbers to sort, apply the following steps:

- Divide the sequence of length n into
- Recursively
- Combine

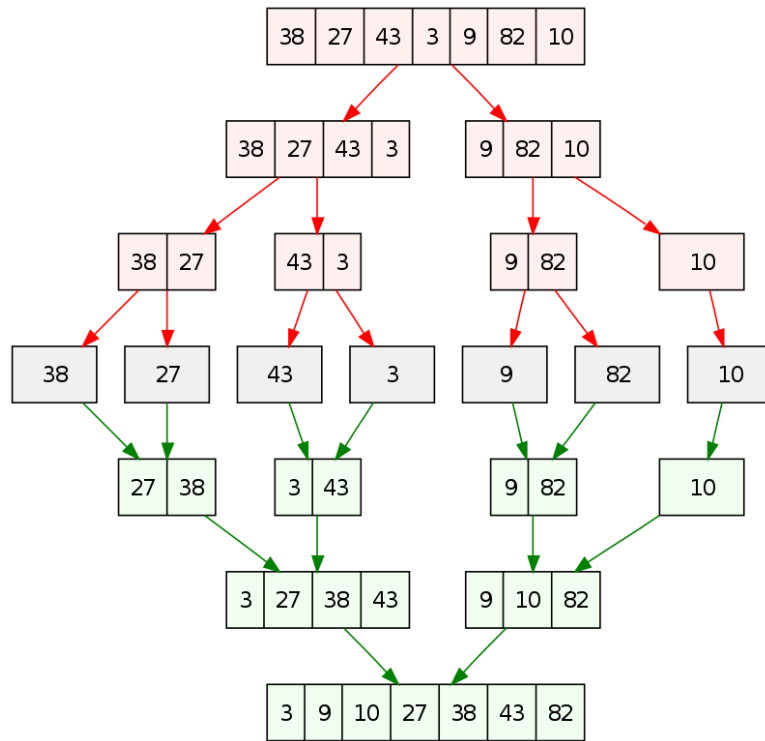


Image courtesy of Wikipedia: https://en.wikipedia.org/wiki/Merge_sort.

```

MERGESORT( $A$ )
   $n = \text{length}(A)$ 
  if  $n == 1$  then

  else
     $m = \lfloor n/2 \rfloor$ 

  end if

```

4.1 Analyzing The Merge Procedure

Correctness: To merge two sorted subarrays into a master array

- Maintain a pointer to the _____
- At each step, _____
- Since subarrays are sorted, one of these numbers is _____

- so place _____
- At each step, we guarantee that: _____
- So continuing until both subarrays are empty, _____

5 Continued Analysis of Merge Sort

Merge Sort. Given n numbers to sort

- Divide the sequence of length n into arrays of length $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$
- Recursively sort the two halves
- (Merge Procedure) Combine the two halves by sorting them.

Question 2. What is the runtime of the *merge procedure* in Merge Sort?

- A** $\Theta(1)$
- B** $\Theta(n \lg n)$
- C** $\Theta(n)$
- D** $\Theta(n^2)$

6 Recurrence Analysis for Divide and Conquer

Runtimes for divide and conquer algorithms can be described in terms of a _____

relation, which _____

Let $T(n)$ denote the runtime for a problem of size n .

Example: merge-sort Assume for this analysis that $n = 2^p$ where $p \in \mathbb{N}$.

7 Three methods for solving recurrences

Given a recurrence relation, there are three approaches to finding the overall runtime.

- **Recursion tree:**
- **Substitution method:**
- **Master theorem:**

8 The Master Theorem for Recurrence Relations

Theorem 8.1. *Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the relation:*

1. *If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then* _____

2. *If $f(n) = \Theta(n^{\log_b a})$, then* _____

3. *If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then* _____

8.1 Example: Merge-Sort

Recall that Merge-Sort satisfies the following recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases} \quad (1)$$

We can apply the master theorem with:

8.2 What to know about the master method?

Proof idea:

A full proof can be found in Section 4.6 of the textbook.

What's more important:

8.3 Examples

Apply the master theorem to the following recurrences:

$$T(n) = 9T(n/3) + n \tag{2}$$

$$T(n) = 3T(n/4) + n \log n \tag{3}$$

$$T(n) = 7T(n/2) + \Theta(n^2) \tag{4}$$